

ODC-GUINÉE

Formation Web Full-Stack — 8 avril au 25 juin 2026

Cahier des Charges Fonctionnel et Technique

MedConnect ODC — Plateforme de Prise de Rendez-vous Médical Sécurisée

Commanditaire	Orange Digital Center Guinée (ODC-GUINÉE)
Projet	Formation Web Full-Stack — Projet de Fin de Formation
Période	8 avril 2026 – 25 juin 2026
Technologies	React 18 / TypeScript · Spring Boot 3 / Java · MySQL 8

Table des Matières

1. **Présentation Générale du Projet**
2. **Parties Prenantes et Acteurs**
3. **Périmètre Fonctionnel Détaillé**
4. **Exigences Non Fonctionnelles**
5. **Architecture Technique Globale**
6. **Synthèse du Modèle de Données**
7. **Référentiel Complet des Endpoints API REST**
8. **Parcours Utilisateurs Détaillés**
9. **Spécifications de l'Interface Utilisateur**
10. **Sécurité de la Plateforme**
11. **Livrables et Critères d'Évaluation**
12. **Planning et Déroulement du Projet**
13. **Guide de Déploiement**
14. **Évolutions Futures**
15. **Glossaire**
16. **Conclusion Générale**

1. Présentation Générale du Projet

1.1. Contexte et genèse du projet

L'accès aux soins de santé en Guinée est confronté à de nombreux défis organisationnels. La prise de rendez-vous médicaux repose encore largement sur des processus manuels, des appels téléphoniques et des registres papier, entraînant des pertes d'information, des double-réservations, des rendez-vous manqués et une absence totale de traçabilité du parcours patient.

Dans ce contexte, l'Orange Digital Center Guinée (ODC-GUINÉE), dans le cadre de sa formation Web Full-Stack (8 avril – 25 juin 2026), a mandaté les équipes de développement en formation pour concevoir et réaliser MedConnect ODC : une plateforme numérique complète destinée à moderniser le système de gestion des rendez-vous médicaux, de la prise de contact entre patient et médecin jusqu'à la dématérialisation du dossier médical.

1.2. Présentation de la solution

MedConnect ODC est une application web full-stack accessible depuis tout navigateur moderne sur ordinateur, tablette et smartphone. Elle met en relation trois types d'acteurs — patients, médecins et administrateurs — via une interface unifiée, sécurisée et adaptée aux usages mobiles. Elle couvre l'intégralité du parcours de soin numérique :

- La recherche et la découverte de médecins disponibles par spécialité.
- La consultation et la réservation en ligne de créneaux médicaux.
- Le suivi du statut des rendez-vous (en attente, confirmé, annulé, terminé).
- La rédaction et la conservation du dossier médical numérique (diagnostic, notes, ordonnances).
- La gestion administrative centralisée des comptes et de l'activité globale.
- Un système de notifications en temps réel pour tenir chaque acteur informé.

1.3. Objectifs généraux

Réf.	Objectif	Indicateur de succès
OBJ-01	Digitaliser la prise de RDV	100 % des réservations gérées via la plateforme.
OBJ-02	Sécuriser les accès	Authentification JWT, rôles distincts, 0 accès non autorisé.
OBJ-03	Dématérialiser le dossier médical	Toute consultation génère une fiche numérique archivée.
OBJ-04	Optimiser la gestion des disponibilités	Le médecin déclare ses créneaux, les conflits sont impossibles.

Réf.	Objectif	Indicateur de succès
OBJ-05	Assurer la traçabilité	Audit log des actions sensibles, historique complet par utilisateur.
OBJ-06	Garantir l'accessibilité mobile	Interface utilisable sur smartphone, tablette et desktop.

2. Parties Prenantes et Acteurs

2.1. Commanditaire et maître d'ouvrage

L'Orange Digital Center Guinée (ODC-GUINÉE) est le commanditaire institutionnel du projet. En tant que hub d'innovation numérique et centre de formation au code de référence, l'ODC encadre le projet et fixe les exigences fonctionnelles, les critères d'évaluation et les livrables attendus pour la soutenance finale.

2.2. Équipe de développement (maître d'œuvre)

Prénom	Nom	Contact	Rôle principal
Nafissatou	Diallo	224 629 417 612	Frontend / Intégration UI
Djamilatou	Diallo	224 629 327 420	Frontend / Design & UX
Mamadou Ramadan	Bah	224 629 349 794	Backend / API REST
Mamadou	Barry	224 624 064 783	Backend / Sécurité & JWT
Sékou	Touré	224 621 107 773	Base de données / DevOps
Amara	Bangoura	224 611 421 367	Chef de projet / Documentation

2.3. Utilisateurs finaux

Profil utilisateur	Description et besoins
Patient	Toute personne cherchant à consulter un médecin. Besoins : trouver un médecin disponible, réserver et gérer ses rendez-vous, accéder à son dossier médical.
Médecin	Praticien de santé enregistré sur la plateforme. Besoins : déclarer ses disponibilités, gérer son agenda, rédiger des rapports de consultation.
Administrateur	Gestionnaire de la plateforme. Besoins : gérer les comptes, modérer l'activité, créer les comptes médecin, superviser l'ensemble du système.

3. Périmètre Fonctionnel Détaillé

Cette section décrit l'ensemble des fonctionnalités attendues, organisées par module fonctionnel et par acteur. Chaque exigence fonctionnelle est référencée pour faciliter le suivi de réalisation.

3.1. Module Authentification & Gestion des Comptes

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-AUTH-01	Inscription d'un patient (formulaire public)	Patient	ESSENTIELLE
F-AUTH-02	Connexion par email + mot de passe	Patient, Médecin	ESSENTIELLE
F-AUTH-03	Connexion administrateur avec clé secrète	Admin	ESSENTIELLE
F-AUTH-04	Déconnexion sécurisée (invalidation du token local)	Tous	ESSENTIELLE
F-AUTH-05	Réinitialisation du mot de passe par email	Tous	IMPORTANTE
F-AUTH-06	Désactivation d'un compte par l'administrateur	Admin	ESSENTIELLE
F-AUTH-07	Réactivation d'un compte suspendu	Admin	IMPORTANTE
F-AUTH-08	Création d'un compte médecin (réservée à l'admin)	Admin	ESSENTIELLE

3.2. Module Gestion des Disponibilités

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-DISPO-01	Déclaration d'un créneau de disponibilité (date, heure début, heure fin)	Médecin	ESSENTIELLE
F-DISPO-02	Calcul automatique de la durée du créneau	Système	ESSENTIELLE
F-DISPO-03	Consultation de ses propres créneaux déclarés	Médecin	ESSENTIELLE
F-DISPO-04	Consultation des créneaux libres par spécialité	Patient	ESSENTIELLE

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-DISPO-05	Blocage automatique d'un créneau lors d'une réservation	Système	ESSENTIELLE
F-DISPO-06	Remise en disponibilité d'un créneau lors d'une annulation	Système	ESSENTIELLE

3.3. Module Gestion des Rendez-vous

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-RDV-01	Réservation d'un créneau libre (prise de RDV)	Patient	ESSENTIELLE
F-RDV-02	Consultation de son planning de RDV personnel	Patient, Médecin	ESSENTIELLE
F-RDV-03	Confirmation d'un rendez-vous par le médecin	Médecin	ESSENTIELLE
F-RDV-04	Annulation d'un rendez-vous par le patient	Patient	ESSENTIELLE
F-RDV-05	Annulation d'un rendez-vous par le médecin	Médecin	ESSENTIELLE
F-RDV-06	Clôture d'un rendez-vous (statut TERMINE)	Médecin	ESSENTIELLE
F-RDV-07	Surveillance globale de l'ensemble des RDV	Admin	ESSENTIELLE
F-RDV-08	Filtrage des RDV par statut, médecin ou patient	Admin, Médecin	IMPORTANTE

3.4. Module Dossier Médical et Consultations

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-CONS-01	Rédaction du rapport de consultation (diagnostic, notes, ordonnance)	Médecin	ESSENTIELLE
F-CONS-02	Consultation de son historique médical complet	Patient	ESSENTIELLE
F-CONS-03	Consultation des fiches de consultation rédigées	Médecin	IMPORTANTE
F-CONS-04	Archivage définitif et immuable	Système	ESSENTIELLE

Réf.	Fonctionnalité	Acteur(s)	Priorité
	des fiches médicales		

3.5. Module Notifications

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-NOTIF-01	Réception de notifications applicatives (confirmation, annulation, rappel)	Patient, Médecin	IMPORTANTE
F-NOTIF-02	Compteur de notifications non lues (icône cloche)	Tous	IMPORTANTE
F-NOTIF-03	Marquage des notifications comme lues (unitaire ou global)	Tous	UTILE

3.6. Module Profil Utilisateur

Réf.	Fonctionnalité	Acteur(s)	Priorité
F-PROF-01	Modification des informations personnelles (nom, prénom, téléphone, adresse)	Tous	IMPORTANTE
F-PROF-02	Changement du mot de passe (après vérification de l'ancien)	Tous	IMPORTANTE
F-PROF-03	Upload et affichage d'une photo de profil	Tous	UTILE

4. Exigences Non Fonctionnelles

4.1. Performance

- Temps de chargement des listes (créneaux, rendez-vous, utilisateurs) : inférieur à 2 secondes.
- Pagination recommandée côté backend pour toutes les listes pouvant dépasser 20 éléments.
- Build frontend optimisé (code splitting, lazy loading des routes non critiques).
- Requêtes SQL optimisées par index ciblés sur les colonnes de filtrage et de tri.

4.2. Sécurité

- Aucun mot de passe stocké en clair — hachage systématique via BCrypt (coût ≥ 10 rounds).
- Authentification sans état (stateless) basée sur JWT, tokens signés avec une clé secrète forte (minimum 32 caractères).
- Contrôle d'accès par rôle (RBAC) enforced côté serveur — aucune règle de sécurité ne repose uniquement sur le frontend.
- Protection CSRF désactivée explicitement au profit du mécanisme JWT (incompatible avec les attaques CSRF par nature).
- CORS configuré pour n'autoriser que l'origine du frontend de production.
- Jetons de réinitialisation de mot de passe à usage unique et avec expiration stricte.
- HTTPS obligatoire en production pour chiffrer le transport des jetons JWT.

4.3. Disponibilité et fiabilité

- Disponibilité cible : 99 % (hors maintenance planifiée).
- Déploiement Docker pour faciliter le redémarrage automatique en cas de crash.
- `spring.jpa.hibernate.ddl-auto=validate` en production pour prévenir toute modification accidentelle du schéma.
- Sauvegarde régulière de la base de données recommandée (dump MySQL quotidien).

4.4. Maintainabilité

- Code TypeScript côté frontend : `typescript` strict pour détecter les erreurs à la compilation.
- Architecture en couches côté backend : Controller / Service / Repository / Model, chaque couche testable indépendamment.
- Composants React réutilisables et découplés, organisés par domaine fonctionnel.
- Documentation des endpoints via des commentaires Javadoc et un potentiel export OpenAPI/Swagger.

- Convention de nommage cohérente en français pour les entités métier, en anglais pour le code Java et TypeScript.

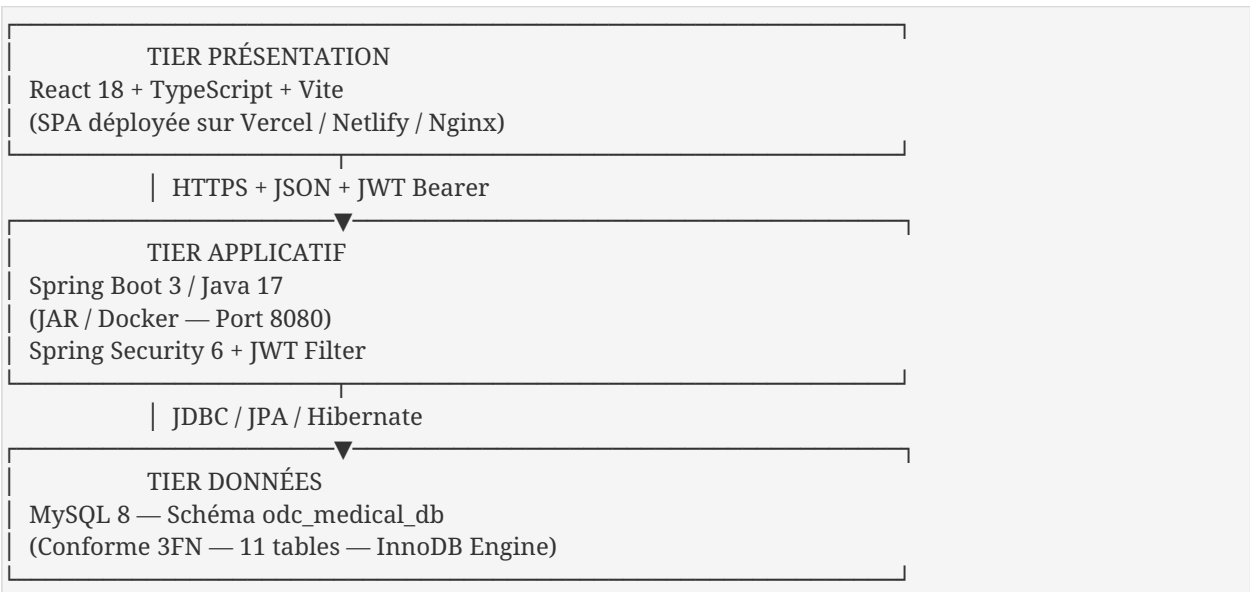
4.5. Accessibilité et internationalisation

- Interface entièrement en langue française.
- Design responsive mobile-first (priorité aux écrans < 768 px).
- Respect des contrastes de couleurs pour les utilisateurs malvoyants (ratio ≥ 4.5:1 recommandé).
- Libellés explicites sur tous les boutons et champs de formulaire (accessibilité lecteurs d'écran).

5. Architecture Technique Globale

5.1. Vue d'ensemble

MedConnect ODC repose sur une architecture découplée en trois tiers distincts, communiquant entre eux via des protocoles standardisés :



5.2. Stack technologique complète

Couche / Domaine	Technologie retenue et justification
Frontend — Framework	React 18 : bibliothèque UI dominante, écosystème mature, composants fonctionnels et hooks.
Frontend — Langage	TypeScript : typage statique réduisant les bugs à la compilation et améliorant la maintenabilité.

Couche / Domaine	Technologie retenue et justification
Frontend — Build	Vite : bundler nouvelle génération, HMR ultra-rapide, build optimisé (tree-shaking, code splitting).
Frontend — Style	Tailwind CSS : utilitaires CSS-first permettant un design cohérent sans feuille de style globale.
Frontend — Composants	shadcn/ui : composants accessibles et personnalisables (Dialog, Toast, Select, Button...).
Frontend — Routing	React Router v6 : gestion déclarative et type-safe des routes SPA.
Frontend — Data fetching	TanStack Query : cache intelligent, états de chargement, dédoublement des requêtes.
Frontend — HTTP Client	Axios : intercepteurs pour injection JWT et gestion centralisée des erreurs.
Backend — Framework	Spring Boot 3 : auto-configuration, démarrage rapide, production-ready (Actuator, logs).
Backend — Langage	Java 17 : LTS, enregistrements, lambdas, switch expressions, performances JIT.
Backend — Sécurité	Spring Security 6 + JWT : stateless, filtre d'authentification, RBAC par rôle.
Backend — Persistance	Spring Data JPA / Hibernate : ORM, requêtes dérivées, transactions déclaratives.
Backend — Build	Maven : gestion des dépendances, cycle de build standardisé, génération du JAR.
Base de données	MySQL 8 : SGBDR éprouvé, transactions ACID, InnoDB, partage d'hébergement aisé.
Contrôle de version	Git + GitHub : branches feature, pull requests, historique des commits.
Déploiement	Docker (optionnel) + Vercel/Netlify (frontend) + VPS ou Render (backend).

6. Synthèse du Modèle de Données

La base de données odc_medical_db est composée de 11 tables relationnelles, organisées autour d'une entité centrale utilisateurs qui centralise l'authentification et est spécialisée par héritage en profils médecin et patient.

Table	Attributs principaux	Rôle
utilisateurs	id, nom, prenom, email, mot_de_passe, id_role, est_actif	Entité centrale d'authentification — commune à tous les rôles.
roles	id_role, nom (ENUM)	Référentiel des rôles : PATIENT, MEDECIN, ADMIN.
specialites	id_specialite, nom, description	Référentiel des spécialités médicales des praticiens.
patients	id_patient, id_utilisateur (FK), telephone, antecedents	Profil métier du patient (héritage 1:1).
medecins	id_medecin, id_utilisateur (FK), id_specialite (FK)	Profil métier du médecin (héritage 1:1).
disponibilites	id_dispo, id_medecin (FK), date_debut, date_fin, est_libre	Créneaux horaires ouverts par les médecins.
rendez_vous	id_rdv, id_patient (FK), id_dispo (FK), statut, motif	Réservations associant un patient à un créneau.
consultations	id_consultation, id_rdv (FK UNIQUE), diagnostic, ordonnance	Fiche médicale 1:1 avec un rendez-vous honoré.
notifications	id_notification, id_utilisateur (FK), message, type, est_lu	Alertes applicatives par utilisateur.
logs	id_log, id_utilisateur (FK), action, details, ip_address	Journal d'audit des actions sensibles.
password_reset_tokens	id, user_id (FK), token, expiry_date	Jetons de réinitialisation de mot de passe.

7. Référentiel Complet des Endpoints API REST

L'ensemble des endpoints exposés par le backend Spring Boot sont documentés ci-après. La sécurité est appliquée côté serveur — le préfixe de la route détermine la vérification du rôle JWT requise.

7.1. Authentification (Public — Aucun JWT requis)

Méthode	Endpoint	Description
POST	/api/auth/register	Inscription d'un nouveau patient. Corps : nom, prénom, email, mot de passe.
POST	/api/auth/login	Connexion (Patient / Médecin). Retourne : JWT + rôle + email.
POST	/api/auth/admin/login	Connexion administrateur. Corps : email, mot de passe, secretKey.
POST	/api/auth/forgot-password	Envoi d'un email de réinitialisation. Corps : email.
POST	/api/auth/reset-password	Réinitialise le mot de passe avec le token email.

7.2. Espace Patient (JWT requis — rôle PATIENT)

Méthode	Endpoint	Description
GET	/api/patient/medecins	Liste des médecins disponibles (filtrable par spécialité).
GET	/api/patient/disponibilites	Créneaux libres (global ou par médecin via query param).
GET	/api/patient/disponibilites/{idMedecin}	Créneaux libres d'un médecin spécifique.
POST	/api/patient/rendez-vous	Réservation d'un créneau. Corps : id_dispo, motif.
GET	/api/patient/rendez-vous	Liste de ses propres rendez-vous.
PATCH	/api/patient/rendez-vous/{id}/annuler	Annulation d'un rendez-vous en attente.
GET	/api/patient/historique	Historique complet des consultations passées.

7.3. Espace Médecin (JWT requis — rôle MEDECIN)

Méthode	Endpoint	Description
POST	/api/medecin/disponibilites	Déclarer un créneau. Corps : date_debut, date_fin.
GET	/api/medecin/disponibilites	Liste de ses propres créneaux déclarés.
GET	/api/medecin/rendez-vous	Agenda du médecin (liste de ses patients attendus).
PATCH	/api/medecin/rendez-vous/{id}/confirmer	Confirmation d'un rendez-vous en attente.
PATCH	/api/medecin/rendez-vous/{id}/annuler	Annulation d'un rendez-vous.
POST	/api/medecin/rendez-vous/{id}/consultation	Rédaction et clôture : diagnostic, notes, ordonnance.

7.4. Espace Administration (JWT requis — rôle ADMIN)

Méthode	Endpoint	Description
GET	/api/admin/users	Liste complète de tous les comptes utilisateurs de la plateforme.
POST	/api/admin/medecins	Création d'un compte médecin avec spécialité obligatoire.
PATCH	/api/admin/users/{id}/desactiver	Désactivation d'un compte (bloqué, données historisées).
PATCH	/api/admin/users/{id}/activer	Réactivation d'un compte suspendu.
GET	/api/admin/rendez-vous	Supervision en temps réel du planning global.

7.5. Endpoints transverses (JWT requis — tout rôle)

Méthode	Endpoint	Description
GET	/api/notifications	Notifications de l'utilisateur connecté (triées par date décroissante).
PATCH	/api/notifications/read-all	Marquer toutes les notifications de l'utilisateur connecté comme lues.
GET	/api/users/profile	Récupération du profil complet de l'utilisateur connecté.
PUT	/api/users/profile	Mise à jour du profil (coordonnées, photo de

Méthode	Endpoint	Description
		profil).
PUT	/api/users/change-password	Changement du mot de passe (vérification de l'ancien requis).

8. Parcours Utilisateurs Détaillés

8.1. Parcours Patient — Prise de rendez-vous

- 1. L'utilisateur accède à la page d'accueil publique (/).
- 2. Il clique sur 'Inscription' et remplit le formulaire (nom, prénom, email, mot de passe).
- 3. Après validation côté backend, un token JWT est retourné — l'utilisateur est redirigé vers son dashboard.
- 4. Depuis le dashboard ou 'Trouver un Médecin', il recherche un médecin par spécialité.
- 5. Il consulte les créneaux disponibles et sélectionne le créneau souhaité.
- 6. Une modal de confirmation s'ouvre : il saisit un motif de consultation et confirme.
- 7. Le rendez-vous est créé en statut ATTENTE. Une notification est envoyée au médecin.
- 8. Depuis 'Mes Rendez-vous', il suit le statut de sa réservation.
- 9. Après la consultation, il retrouve son diagnostic et son ordonnance dans 'Dossier Médical'.

8.2. Parcours Médecin — Gestion de l'agenda

- L'administrateur crée le compte du médecin en lui attribuant une spécialité. Un email de connexion lui est communiqué.
- Le médecin se connecte via /login avec ses identifiants — son JWT encode le rôle MEDECIN.
- Depuis son dashboard, il accède à 'Mes Disponibilités' et déclare ses créneaux (date, heure début, heure fin).
- Dès qu'un patient réserve un créneau, le médecin reçoit une notification et voit le rendez-vous apparaître dans son agenda.
- Il confirme ou annule le rendez-vous depuis la page 'Mon Agenda'.
- À l'issue de la consultation, il ouvre la fiche du rendez-vous et rédige le rapport médical (diagnostic, notes, ordonnance).
- La fiche est archivée définitivement et devient accessible au patient dans son dossier médical.

8.3. Parcours Administrateur — Supervision

- L'administrateur accède à /login (onglet Admin) et saisit ses identifiants + clé secrète.
- Depuis son dashboard, il consulte la liste des utilisateurs (onglet Utilisateurs).
- Il crée les comptes médecins en leur attribuant une spécialité depuis le formulaire dédié.
- Il peut activer ou désactiver tout compte depuis les actions de la liste.
- L'onglet Rendez-vous lui permet de surveiller l'ensemble du planning en temps réel.

9. Spécifications de l'Interface Utilisateur

9.1. Pages et fonctionnement de chaque écran

Écran / Route	Description de l'interface
/ (Accueil)	Hero section avec les logos ODC + MedConnect, présentation de la plateforme, recherche rapide de médecins, statistiques clés (nombre de médecins, spécialités, patients enregistrés), call-to-action Inscription.
/login	Formulaire de connexion avec deux onglets : Espace Patient/Médecin et Espace Admin (avec champ clé secrète supplémentaire). Lien vers /signup et 'Mot de passe oublié'.
/signup	Formulaire d'inscription patient avec validation en temps réel (email valide, mot de passe ≥ 8 caractères, champs requis). Lien de retour vers /login.
/patient (Dashboard)	4 cartes statistiques (RDV à venir, consultations passées, médecins consultés, notifications non lues). Liste des prochains RDV. Boutons d'accès rapide.
/find-doctors	Barre de recherche + filtre par spécialité. Grille de cartes médecin avec photo, nom, spécialité, étoile de notation et bouton 'Prendre RDV'. Modal de sélection du créneau et de confirmation.
/appointments (Patient)	Liste tabulaire des RDV avec colonnes : médecin, spécialité, date/heure, statut (badge coloré), actions (annuler si applicable).
/records	Historique des consultations sous forme de cartes dépliantes : date, médecin, diagnostic, notes, ordonnance.
/doctor (Dashboard)	Statistiques du jour : RDV confirmés, en attente, annulés. Liste des patients attendus dans la journée. Bouton rapide 'Déclarer une disponibilité'.
/appointments (Médecin)	Tableau de l'agenda avec filtre par statut. Pour chaque RDV : patient, date/heure, motif, actions (confirmer / annuler / rédiger rapport). Modal de saisie du rapport.

Écran / Route	Description de l'interface
/admin (Dashboard)	Interface à deux onglets. Onglet Utilisateurs : tableau de tous les comptes avec boutons activer/désactiver et formulaire de création de médecin. Onglet Rendez-vous : tableau global du planning.
/settings	Formulaire de profil (photo, coordonnées) et section changement de mot de passe.

9.2. Directives design (Design System ODC)

- Palette : Orange ODC (#E8590C) pour les actions principales et la marque, Bleu profond (#1B4F72) pour la confiance médicale et les titres, Blanc pour les fonds de cartes et les formulaires.
- Typographie : police Inter ou Poppins (Google Fonts) — sans-serif, lisible sur toutes les tailles d'écran.
- Iconographie : icônes Lucide React cohérentes dans toute l'application (calendrier, utilisateur, cloche, médecin, etc.).
- Composants : boutons, champs, modals et toasts issus de shadcn/ui pour une cohérence visuelle totale et une accessibilité WCAG intégrée.
- Responsive : breakpoints Tailwind CSS — mobile (< 768 px), tablette (768–1024 px), desktop (> 1024 px).

10. Sécurité de la Plateforme

10.1. Authentification et autorisation

La sécurité de MedConnect ODC est implémentée à plusieurs niveaux complémentaires :

Mécanisme	Description
BCrypt Password Hashing	Tous les mots de passe sont hachés avec BCrypt (facteur de coût ≥ 10) avant d'être persistés en base. La vérification se fait en comparaison de hash, jamais en clair.
JWT Bearer Token	Chaque utilisateur authentifié reçoit un jeton JWT signé contenant ses claims (userId, email, rôle). Le jeton est transmis dans l'en-tête Authorization de chaque requête.
JwtAuthenticationFilter	Filtre Spring Security interceptant chaque requête : extraction, vérification de signature, vérification d'expiration, injection dans le SecurityContext.

Mécanisme	Description
Contrôle d'accès par rôle (RBAC)	Chaque famille d'endpoints (/api/patient/**, /api/medecin/**, /api/admin/**) est protégée par une exigence de rôle vérifiée côté serveur.
Clé secrète administrateur	L'accès au compte Admin nécessite un second facteur : la saisie d'une clé secrète globale vérifiée côté backend, en plus des identifiants classiques.
CSRF désactivé (intentionnel)	L'API REST JWT étant stateless et n'utilisant pas de cookies, la protection CSRF classique est inutile et désactivée explicitement dans SecurityFilterChain.
CORS configuré	Seule l'origine du frontend de production est autorisée dans la configuration CORS du backend.
Tokens de reset à usage unique	Les jetons de réinitialisation de mot de passe sont invalidés après utilisation et possèdent une expiration stricte.

10.2. Sécurité de la base de données

- Aucune valeur sensible (mot de passe, clé secrète) n'est stockée en clair en base de données.
- Les contraintes d'intégrité référentielle (FOREIGN KEY) et les contraintes CHECK sont enforcées par le SGBD, indépendamment de la couche applicative.
- La table logs assure l'audit des actions sensibles (connexion, création de compte, désactivation).
- Le compte de base de données utilisé par l'application doit avoir des droits minimaux (SELECT, INSERT, UPDATE, DELETE) — aucun droit DDL en production.

11. Livrables et Critères d'Évaluation

11.1. Livrables du projet

Réf.	Livrable	Statut
L-01	Code source Frontend (React/TypeScript) — dépôt GitHub	✓ Livré
L-02	Code source Backend (Spring Boot/Java) — dépôt GitHub	✓ Livré
L-03	Script SQL d'initialisation de la base de données	✓ Livré
L-04	Cahier des Charges (ce document)	✓ Livré

Réf.	Livrable	Statut
L-05	Cahier de Conception — Base de données	✓ Livré
L-06	Cahier de Conception — Backend	✓ Livré
L-07	Cahier de Conception — Frontend	✓ Livré
L-08	Présentation PowerPoint (soutenance)	✓ Livré
L-09	Application déployée et accessible en ligne	✓ Livré
L-10	Export de la base de données (dump MySQL)	✓ Livré

11.2. Critères d'évaluation ODC

- Fonctionnalités (respect des besoins, workflows complets, cohérence des parcours) : vérification de la couverture de toutes les exigences fonctionnelles listées en section 3.
- Qualité technique (backend propre, base de données bien structurée, bonnes pratiques de code) : architecture en couches, normalisation 3FN, sécurité JWT, contrôle d'accès par rôle.
- UI & Professionnalisme (interface, expérience utilisateur, sécurité, bonus et présentation) : respect du design system ODC, responsive, accessibilité, messages d'erreur clairs.

12. Planning et Déroulement du Projet

La formation et le projet se sont déroulés du 8 avril au 25 juin 2026 sous la supervision de l'ODC-Guinée. La présentation finale dure 10 à 15 minutes par groupe (y compris la session de questions-réponses).

Phase	Contenu principal	Période estimée
Phase 1 — Analyse	Étude du cahier des charges, identification des exigences, modélisation du MCD.	Jours 1–2
Phase 2 — Conception	Élaboration du MLD et du MPD, rédaction du script SQL, design de l'architecture logicielle.	Jours 3–4
Phase 3 — Développement Backend	Implémentation des entités JPA, repositories, services et controllers. Sécurité JWT.	Jours 5–7
Phase 4 — Développement	Intégration de l'UI React, routing, formulaires, appels API, dashboards.	Jours 6–7

Phase	Contenu principal	Période estimée
Frontend		
Phase 5 — Intégration & Tests	Tests d'intégration frontend/backend, correction de bugs, harmonisation des endpoints.	Jour 8
Phase 6 — Déploiement & Documentation	Déploiement en ligne, rédaction des cahiers de conception, préparation de la soutenance.	Jour 9 - 10

13. Guide de Déploiement

13.1. Déploiement du backend Spring Boot

```
# 1. Compilation du projet
mvn clean package -DskipTests

# 2. Lancement avec les variables d'environnement
java -jar target/medconnect-backend.jar \
  --spring.datasource.url=jdbc:mysql://HOST:3306/odc_medical_db \
  --spring.datasource.username=DB_USER \
  --spring.datasource.password=DB_PASS \
  --jwt.secret=CLE_SECRETE_FORTE_MINIMUM_32_CHARS \
  --admin.secret.key=CLE_ADMIN_GLOBALE

# 3. Ou via Docker
docker build -t medconnect-backend .
docker run -p 8080:8080 --env-file .env medconnect-backend
```

13.2. Déploiement du frontend React

```
# 1. Configuration de l'URL de l'API
# Fichier .env.production
VITE_API_BASE_URL=https://api.medconnect-odc.com/api

# 2. Build de production
npm run build
# → Génère le dossier dist/ avec les assets optimisés

# 3. Déploiement sur Vercel (recommandé)
vercel --prod
```

13.3. Initialisation de la base de données

```
# 1. Connexion MySQL
mysql -u root -p

# 2. Exécution du script d'initialisation
source /chemin/vers/script.sql

# 3. Données de référence à insérer manuellement
INSERT INTO roles (nom) VALUES ('PATIENT'), ('MEDECIN'), ('ADMIN');
INSERT INTO specialites (nom) VALUES
('Cardiologie'), ('Pédiatrie'), ('Dermatologie'),
('Médecine générale'), ('Gynécologie'), ('Neurologie');
```

14. Évolutions Futures

- Messagerie interne : système de messages directs entre patient et médecin, actuellement un placeholder dans l'application (/messages).
- Téléconsultation : intégration d'un module de visioconférence (WebRTC ou API tierce) pour les consultations à distance.
- Paiement en ligne : intégration d'une passerelle de paiement (Orange Money, Wave) pour la facturation des consultations.
- Système de notation des médecins : permettre aux patients d'évaluer un médecin après une consultation (base de l'affichage des étoiles déjà intégré).
- Application mobile native : React Native utilisant la même API REST pour Android et iOS.
- Rapports analytiques avancés : tableaux de bord administrateur avec graphiques d'activité, taux de remplissage des créneaux, médecins les plus sollicités.
- Internationalisation (i18n) : support du français, de l'anglais et des langues locales guinéennes.
- Intégration e-mail : envoi automatique d'emails de rappel de rendez-vous 24h avant la consultation.

15. Glossaire

Terme	Définition
API REST	Interface de Programmation Applicative respectant le style architectural REST : communication sans état, ressources identifiées par des URLs, verbes HTTP standards (GET, POST, PUT, PATCH, DELETE).
BCrypt	Algorithme de hachage de mots de passe adaptatif et résistant aux attaques par force brute grâce à son facteur de coût configurable.
CORS	Cross-Origin Resource Sharing : mécanisme HTTP permettant à un frontend hébergé sur un domaine d'accéder à une API hébergée sur un autre domaine.
CSRF	Cross-Site Request Forgery : attaque consistant à faire exécuter des actions à un utilisateur authentifié à son insu. Non applicable aux API JWT stateless.
DTO	Data Transfer Object : objet de transfert de données utilisé pour découpler la représentation externe (JSON) des entités internes (JPA).
InnoDB	Moteur de stockage MySQL supportant les transactions ACID et les clés étrangères — utilisé pour toutes les tables de MedConnect ODC.
JPA / Hibernate	Java Persistence API : spécification Java pour le mapping objet-relationnel. Hibernate en est l'implémentation de référence.
JWT	JSON Web Token : standard compact et sécurisé pour transmettre des informations entre parties sous forme de jeton JSON signé numériquement.
RBAC	Role-Based Access Control : contrôle d'accès basé sur les rôles — chaque utilisateur n'accède qu'aux ressources autorisées pour son rôle.
SPA	Single Page Application : application web monopage où la navigation est gérée par JavaScript côté client sans rechargement complet de la page.
Spring Boot	Framework Java simplifiant la création d'applications Spring grâce à l'auto-configuration et à l'intégration d'un serveur embarqué (Tomcat).
3FN	Troisième Forme Normale : niveau de normalisation d'une base de données garantissant l'absence de redondance et de dépendance transitive entre attributs.
TypeScript	Sur-ensemble de JavaScript ajoutant le typage statique optionnel, compilé en JavaScript standard.

Terme	Définition
Vite	Bundler JavaScript nouvelle génération offrant un démarrage de serveur de développement quasi-instantané et un build optimisé pour la production.

Conclusion Générale

MedConnect ODC représente une réponse concrète et techniquement ambitieuse aux enjeux de digitalisation du système de santé guinéen. En s'appuyant sur un stack moderne et éprouvé (React 18 / Spring Boot 3 / MySQL), sur une architecture sécurisée par rôle et une base de données normalisée en 3FN, la plateforme pose les fondations d'un système de gestion médicale robuste, évolutif et accessible.

Au-delà de la démonstration des compétences techniques acquises lors de la formation ODC-Guinée, ce projet illustre la capacité de l'équipe à concevoir une solution numérique complète — de la modélisation des données à l'expérience utilisateur — répondant à un besoin réel de la société guinéenne. Il constitue une base solide pour une mise en production réelle et des évolutions futures ambitieuses.